Практическая работа 3. Атомарные операции в CUDA

CUDA для GPU, начиная с производительности 1.1 и выше, поддерживает атомарные операции над глобальной и разделяемой памятью. Атомарность заключается в том, что гарантируется корректность выполнения операции в случае, когда много нитей пытаются ее выполнить одновременно.

Атомарные арифметические операции

Наиболее часто используемыми атомарными арифметическими операциями являются atomicAdd и atomicSub, служащие для увеличения или уменьшения величины на заданное значение. При этом функция возвращает старое значение. Нужно обратить внимание, что atomicAdd поддерживает операции над 64-битовыми величинами, но только расположенными в глобальной памяти.

```
int atomicAdd( int *addr, int value );
unsigned int atomicAdd( unsigned int *addr, unsigned
int value );

unsigned  long long atomicAdd(unsigned  long long
*addr, unsigned  long long value );

int atomicSub( int *addr, int value );

unsigned  int atomicSub(unsigned  int *addr, unsigned
int value );
```

Операция atomicExch осуществляет атомарный обмен значениями – передаваемое значение записывается по указанному адресу, а предыдущее значение возвращается. При этом подобный обмен происходит как одна транзакция, то есть ни одна нить не может «вклиниться» между шагами этого обмена. Операции над 64-битовыми значениями поддерживаются только для глобальной памяти.

```
int atomicExch( int *addr, int value );

unsigned int atomicExch(unsigned int *addr, unsigned
int value );

unsigned long long atomicExch(unsigned long long
*addr, unsigned long long value );

float atomicExch( float *addr, float value );
```

Следующие две операции сравнивают значение по адресу с переданным значением, записывают минимум/максимум из этих двух значений по заданному адресу и возвращают предыдущее значение, находившееся по адресу. Все эти шаги выполняются атомарно, как одна транзакция.

```
int atomicMin( int *addr, int value );

unsigned int atomicMin(unsigned int *addr, unsigned
int value );

int atomicMax( int *addr, int value );

unsigned int atomicMax(unsigned int *addr, unsigned
int value );
```

Операция atomicInc читает слово по заданному адресу и сравнивает его с переданным значением. Если прочитанное слово больше, то по адресу записывается нуль, иначе значение по адресу увеличивается на единицу. Возвращается старое значение.

```
unsigned int atomicInc(unsigned int *addr, unsigned
int value );
```

Операция atomicDec читает слово по переданному адресу. Если прочитанное значение равно нулю или больше переданного значения, то записывает

по адресу переданное значение, иначе уменьшает значение по адресу на единицу. Возвращается старое значение.

```
unsigned int atomicDec(unsigned int *addr, unsigned
int value );
```

Следующая функция (CAS – Compare And Swap) читает старое 32- или 64-битовое значение по переданному адресу и сравнивает его с параметром compare. В случае совпадения по переданному адресу записывается значение параметра value, иначе значение по адресу не изменяется. Возвращается всегда старое прочитанное значение.

```
int atomicCAS( int *addr, int compare, int value );

unsigned int atomicCAS(unsigned int *addr, unsigned
int compare, unsigned int value );

unsigned long long atomicCAS(unsigned int *addr, un-
signed long long compare, unsigned long long value );
```

Атомарные побитовые операции

Побитовые атомарные операции читают слово по заданному адресу, применяют к нему заданную побитовую операцию с заданным параметром и записывают результат обратно. Возвращается всегда старое значение, находившееся по переданному адресу до начала операции.

```
int atomicAnd( int *addr, int value );

unsigned int atomicAnd(unsigned int *addr, unsigned
int value );

int atomicOr( int *addr, int value );
```

```
unsigned int atomicOr(unsigned int *addr, unsigned
int value );

int atomicXor( int *addr, int value );

unsigned int atomicXor(unsigned int *addr, unsigned
int value );
```

Дополнительные возможности Compute Capability 6.x

Начиная с производительности 6.x, существует ряд нововведений. В частности, представлен новый тип атомарных операций. Например, atomicAdd_system гарантирует, что инструкция является атомарной относительно других CPU и GPU в системе. Функция atomicAdd_block подразумевает, что инструкция является атомарной только с учетом атомистики от других потоков в одном блоке потока. В следующем примере как CPU, так и GPU могут атомарно обновлять целочисленное значение по адресу addr:

```
__global__ void add10(int *addr) {
    // only available on devices with compute
    // capability 6.x
    atomicAdd_system(addr, 10);
}

void foo()
{
    int *addr;
    cudaMallocManaged(&addr, 4);
    *addr = 0;
    add10<<<2, 5>>>(addr);
    // CPU atomic operation
    _sync_fetch_and_add(addr, 10);
}
```

Также в версиях производительности 6.x добавлено сложение 64-битных чисел.

Пример использования атомарных операций

Пусть задана двумерная сетка нитей, и требуется сосчитать количество нитей, индексы которых удовлетворяют некоторому условию. Эту задачу можно реализовать следующим способом: каждая нить вызывает некую булеву функцию, и если результат вызова этой функции равен true, добавляет единицу некой глобальной переменной res.

Приведем ядро такой программы.

```
__global__ void Kernel(int * res) {

  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int j = blockIdx.y * blockDim.y + threadIdx.y;

  if ( Condition(i, j) )// проверка условия
     {
    atomicAdd(res, 1);
  }
}
```

Здесь `Condition(i, j)` некоторое условие, например, `i>=j`.

Практическая работа

1. Вычислите приблизительное значение числа π методом Монте-Карло. Постройте равномерную сетку нитей в квадрате [0, 1]X[0, 1] и посчитайте количество нитей, которые попали в круг с центром в нуле и с радиусом, равным единице. Для вычисления случайного числа напишите __device__ функцию.

Контрольные вопросы

1. Что такое атомарная операция? В чем ее ключевое отличие от обычных операций?

2. Какие атомарные операции вы знаете?